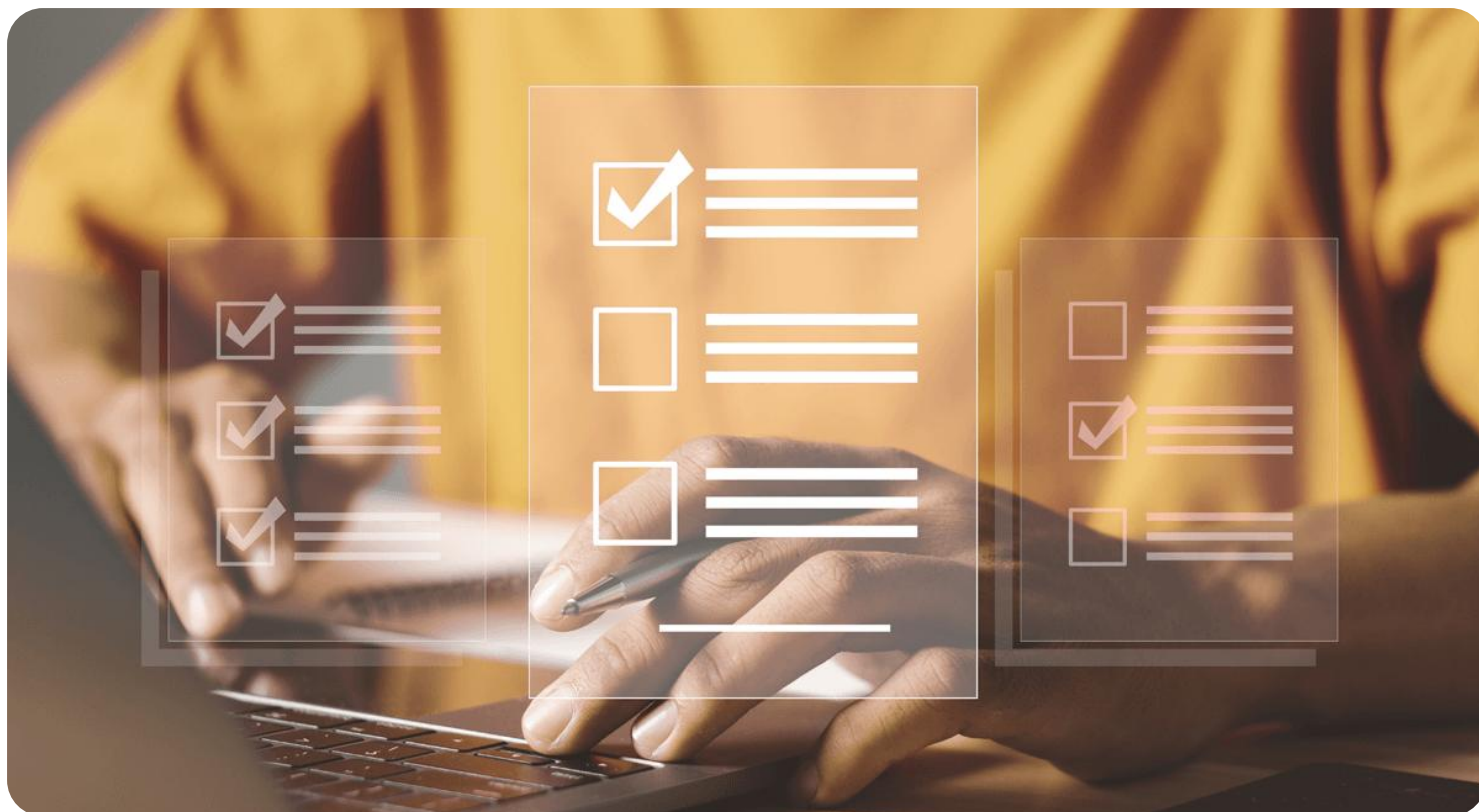




Semana 8

Desarrollo Backend II (PBY2202)

Formato de respuesta

Nombre estudiante: Francisco Henríquez – Raúl Zúñiga	
Asignatura: Desarrollo Backend II	Carrera: Desarrollo de Aplicaciones
Profesor: Jorge Carmona	Fecha: 13-07-2026

Descripción de la actividad

En esta octava semana, realizarás una evaluación sumativa grupal (2 a 3 integrantes), llamada "Implementando de forma avanzada documentación en Microservicios con OpenAPI y HATEOAS". Deberás elaborar de manera detallada la documentación de la arquitectura y los endpoints de los microservicios utilizando OpenAPI Specification (OAS), asegurando la consistencia y alineación con los estándares y requerimientos específicos. Además, deberás implementar HATEOAS en los microservicios, proporcionando enlaces dinámicos que mejoren significativamente la navegabilidad y comprensión de la API.

Instrucciones específicas

Para el desarrollo de la actividad sumativa de la semana, tendrás que analizar el caso desarrollado durante toda la asignatura, y que en esta oportunidad requiere la implementación de microservicios para mejorar su arquitectura de software:

Contexto

El sistema backend de **Minimarket Plus** que vienes desarrollando implementa microservicios para gestionar productos, carritos de compra, inventario, ventas y usuarios con distintos roles. Esta etapa busca **eleva la calidad técnica del backend** documentando los endpoints con OpenAPI e incorporando enlaces dinámicos con HATEOAS, que faciliten la navegación y el trabajo de desarrolladores o integradores externos, fortaleciendo así la operación del negocio y la experiencia del cliente.

Necesidades técnicas

Para cumplir los objetivos del proyecto Minimarket Plus, tu sistema backend debe:

- Documentar los endpoints REST con anotaciones OpenAPI (springdoc-openapi).
- Implementar enlaces dinámicos con HATEOAS usando `EntityModel` o `CollectionModel`.
- Generar la interfaz interactiva Swagger UI y revisar la documentación.
- Exportar el JSON desde `/v3/api-docs` y validarlo importándolo en Postman.
- Mantener la autenticación y roles configurados previamente para proteger los recursos.



Importante

El código de "MINIMARKET PLUS" será proporcionado para su descarga a través del Aula Virtual. Deberás utilizar este backend como base para la actividad, enfocándote en el análisis y configuración del framework de seguridad según los requerimientos proporcionados.

Ahora que has revisado y analizado el caso, tendrás que realizar los siguientes pasos:

Paso 1: Configuración del entorno de pruebas unitarias

1. Configura un proyecto en Spring Boot con Maven o Gradle.
2. Agrega las dependencias necesarias para OpenAPI y HATEOAS
3. Organiza el proyecto con las carpetas estándar:
 - `src/main/java` para el código de las entidades.
 - `src/test/java` para las pruebas unitarias.

Paso 2: Diseño e implementación de endpoints

- Define los controladores para las entidades principales del sistema:
 - Producto: gestión completa de productos.
 - Carrito: operaciones para agregar y eliminar productos del carrito.
 - Inventario: registro y consulta de movimientos de stock.
 - Usuario: administración de clientes y roles.
- Crea las rutas básicas para cada entidad:
 - GET /api/productos: lista todos los productos
 - POST /api/productos: agrega un producto nuevo
 - GET /api/carrito: lista de productos del carrito
 - POST /api/carrito: agrega un producto al carrito
 - GET /api/inventario: lista los movimientos de inventario
 - POST /api/inventario: registra un nuevo movimiento
 - GET /api/usuarios: lista los usuarios registrados
 - POST /api/usuarios: agrega un nuevo usuario
- Anota cada método con OpenAPI, usando `@Operation` y `@ApiResponse`.

Paso 3: Implementación de HATEOAS

- Incluye enlaces dinámicos en las respuestas JSON usando `EntityModel` o `CollectionModel` para guiar la navegación entre recursos.
- Asegúrate de que los enlaces sean claros y funcionales, siguiendo buenas prácticas RESTful.

Paso 4: Validación y pruebas

- Ejecuta la aplicación y accede a la interfaz Swagger UI en <http://localhost:8080/swagger-ui.html>.
- Prueba los endpoints utilizando Postman o Swagger UI, verificando:
 - Respuesta correcta a las solicitudes.
 - Inclusión de enlaces HATEOAS en las respuestas JSON.
 - Consistencia en la documentación generada por OpenAPI.
- Si encuentras errores, revisa y ajusta la lógica o configuración del proyecto.

Paso 5: Documentación y entrega

Redacta directamente en este documento un informe técnico que incluya:

1. Resumen técnico del avance respecto a semanas anteriores

Describe cómo el trabajo de autenticación, pruebas unitarias y documentación inicial en OpenAPI permitió ahora implementar HATEOAS y una documentación avanzada del backend.

2. Análisis de la documentación generada

Explica:

- ¿Qué endpoints se documentaron con más detalle?
- ¿Qué dificultades encontraste al integrar HATEOAS y cómo las solucionaste?
- ¿Qué mejoras realizaste para cumplir los estándares del proyecto?

3. Evidencia de ejecución

Incluye:

- Capturas de Swagger UI mostrando los endpoints con sus ejemplos.
- El JSON exportado desde `/v3/api-docs`.
- Evidencias de las pruebas en Postman.

4. Reflexión técnica

- ¿Cómo aporta OpenAPI junto a HATEOAS a la calidad, mantenibilidad y navegabilidad del backend?
- ¿Qué estrategias usarás para mantener actualizada esta documentación en versiones futuras?

5. Preguntas de apoyo

- ¿Qué entidades necesitaron más trabajo para definir correctamente los enlaces HATEOAS?
- ¿Cómo detectaste y corregiste inconsistencias en los contratos?
- ¿Qué valor agregado entrega esto para los desarrolladores que utilicen el API?

Paso 6: GitHub

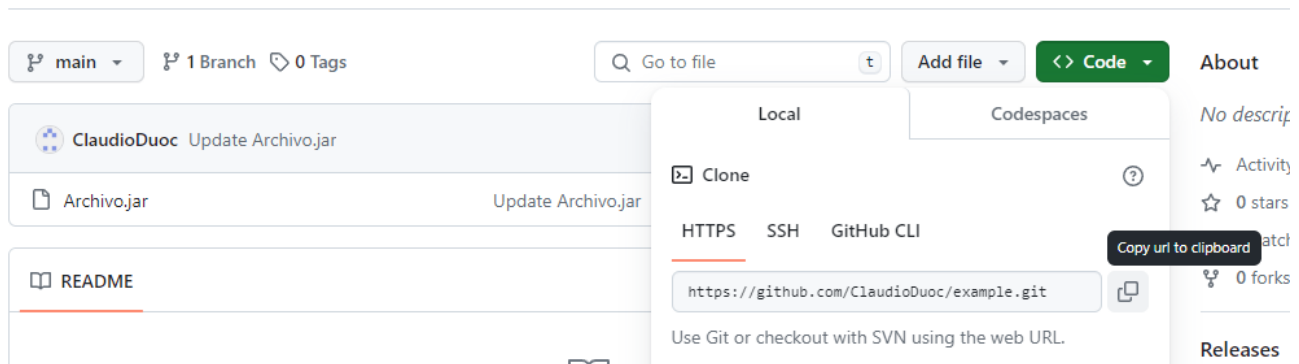
El código deberás subirlo a tu repositorio GitHub. Si no has creado tu cuenta aún, puedes hacerlo a través del siguiente enlace:

<https://github.com/>

Posteriormente, desde el repositorio, deberás generar un enlace de tu proyecto:

Figura 1

Enlace de proyecto GitHub



Nota. Ejemplo genérico de donde se extrae un enlace en GitHub. GitHub (s.f.). *GitHub*.

<https://github.com/>

Paso 7: Entrega

Para tu entrega, no olvides adjuntar este documento con tus entregables y subir al AVA, junto con el enlace de GitHub a adjuntar en la sección “Entrega”.

Entregables

En este apartado deberás adjuntar lo solicitado en los pasos anteriores sobre:

1. Diseño y justificación de los endpoints documentados

- Explica brevemente cómo se diseñaron los endpoints principales (Producto, Carrito, Inventario, Usuario), justificando su alineación con los estándares de OpenAPI Specification (OAS).
- Incluye capturas de la documentación generada en Swagger UI que respalden su implementación.

Durante las semanas anteriores, se consolidó la base funcional del backend de Minimarket Plus, con autenticación con Spring Security, persistencia con JPA, organización por capas y control de acceso a los recursos. Ese trabajo previo fue clave para esta semana, porque permitió documentar endpoints reales sobre una estructura de servicios ya establecidos y protegidos.

Con esa base, el proyecto avanzó hacia una capa más madura de exposición de servicios:

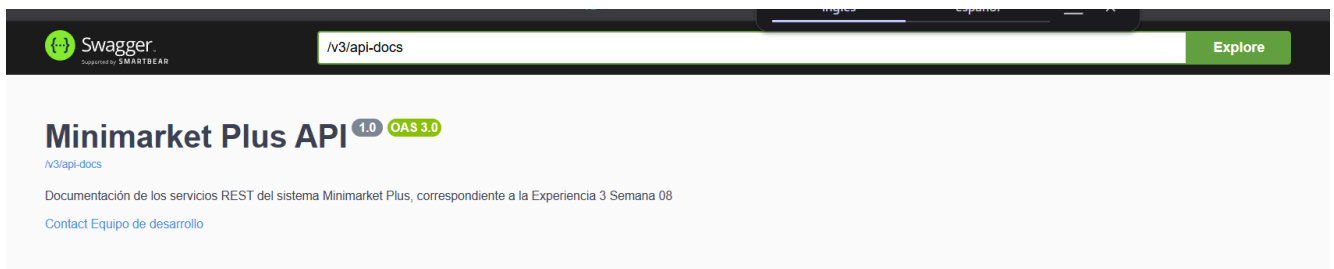
- Se incorporó *springdoc-openapi* a nuestro archivo pom, para generar documentación automática y una interfaz Swagger UI. Para ello se incorporó la dependencia correspondiente.

```
45     <dependency>
46         <groupId>org.springdoc</groupId>
47         <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
48         <version>2.6.0</version>
49     </dependency>
```

- Se ajustó la configuración de seguridad para permitir el acceso a `/swagger-ui.html` y `/v3/api-docs`

```
OpenApiConfig.java
src > main > java > com > minimarket > config > J OpenApiConfig.java > ...

4 import io.swagger.v3.oas.models.info.Contact;
5 import io.swagger.v3.oas.models.info.Info;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
8
9 @Configuration
10 public class OpenApiConfig {
11
12     @Bean
13     public OpenAPI minimarketOpenAPI() {
14         return new OpenAPI()
15             .info(new Info()
16                 .title(title: "Minimarket Plus API")
17                 .version(version: "1.0")
18                 .description(description: "Documentación de los servicios REST del sistema Minimarket Plus, correspondiente a la Experiencia 3 Semana 08")
19                 .contact(new Contact()
20                     .name(name: "Equipo de desarrollo")
21                     .email(email: "rau.zuniga@duocuc.cl")));
22     }
23 }
24
```



- Se anotaron los controladores principales con metadatos OpenAPI (Carrito, Usuarios, Productos y Categorías). Los endpoints principales que se diseñaron siguieron una estructura REST coherente:

GET: para consulta de recursos

POST: para creación

PUT: para actualización

DELETE: para eliminación

La organización por recursos (`/api/productos`, `/api/carrito`, `/api/inventario`, `/api/usuarios`) facilita la lectura del API y se alinea con OpenAPI Specification al exponer rutas claras, verbos HTTP consistentes y descripciones formales de respuesta.

Imagen de la clase CarritoController.java

```
@RestController
@RequestMapping("/api/carrito")
@Tag(name = "Carrito", description = "Endpoints para gestionar los productos agregados al carrito")
public class CarritoController {

    @Autowired
    private CarritoService carritoService;

    @GetMapping
    @Operation(summary = "Listar carrito", description = "Obtiene todos los items del carrito")
    public List<Carrito> listarCarrito() {
        return carritoService.findAll();
    }

    @GetMapping("/{id}")
    @Operation(summary = "Buscar item del carrito por ID", description = "Retorna un item específico del carrito")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Item encontrado"),
        @ApiResponse(responseCode = "404", description = "Item no encontrado")
    })
    public ResponseEntity<Carrito> obtenerCarritoPorId(@PathVariable Long id) {
        Carrito carrito = carritoService.findById(id);
        return (carrito != null) ? ResponseEntity.ok(carrito) : ResponseEntity.notFound().build();
    }
}
```

Imagen de UsuarioController.java

```
src > main > java > com > minimarket > controller > UsuarioController.java > UsuarioController
17 @RestController
18 @RequestMapping("/api/usuarios")
19 @Tag(name = "Usuarios", description = "Endpoints para administrar usuarios del sistema")
20 public class UsuarioController {
21
22     @Autowired
23     private UsuarioService usuarioService;
24
25     @GetMapping
26     @Operation(summary = "Listar usuarios", description = "Obtiene todos los usuarios registrados en el sistema")
27     @ApiResponses({
28         @ApiResponse(responseCode = "200", description = "Lista de usuarios obtenida correctamente"),
29         @ApiResponse(responseCode = "401", description = "No autorizado")
30     })
31     public List<Usuario> listarUsuarios() {
32         return usuarioService.findAll();
33     }
34
35     @GetMapping("/{id}")
36     @Operation(summary = "Buscar usuario por ID", description = "Retorna un usuario específico según su identificador")
37     @ApiResponses({
38         @ApiResponse(responseCode = "200", description = "Usuario encontrado"),
39         @ApiResponse(responseCode = "404", description = "Usuario no encontrado"),
40         @ApiResponse(responseCode = "401", description = "No autorizado")
41     })
42     public ResponseEntity<Usuario> obtenerUsuarioPorId(@PathVariable Long id) {
```

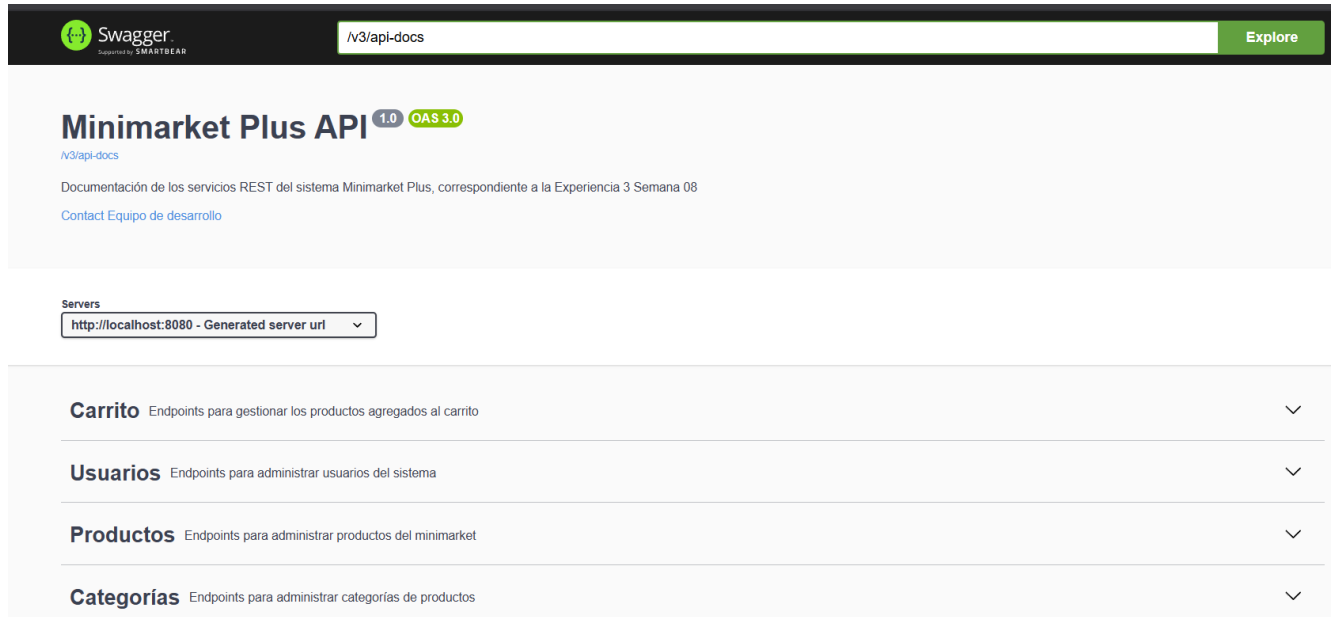
Imagen de ProductosController.java

```
src > main > java > com > minimarket > controller > J ProductoController.java > {} com.minimarket.controller
19
20 import java.util.List;
21
22 @RestController
23 @RequestMapping("/api/productos")
24 @Tag(name = "Productos", description = "Endpoints para administrar productos del minimarket")
25 public class ProductoController {
26
27     @Autowired
28     private ProductoService productoService;
29
30     @GetMapping
31     @Operation(summary = "Listar productos", description = "Obtiene todos los productos registrados")
32     public List<Producto> listarProductos() {
33         return productoService.findAll();
34     }
35
36     @GetMapping("/{id}")
37     @Operation(summary = "Buscar producto por ID", description = "Retorna un producto específico según su identificador")
38     @ApiResponses({
39         @ApiResponse(responseCode = "200", description = "Producto encontrado"),
40         @ApiResponse(responseCode = "404", description = "Producto no encontrado")
41     })
42     public ResponseEntity<Producto> obtenerProductoPorId(@PathVariable Long id) {
43         Producto producto = productoService.findById(id);
44         return (producto != null) ? ResponseEntity.ok(producto) : ResponseEntity.notFound().build();
45     }
46 }
```

Imagen de CategoríasController.java

```
c > main > java > com > minimarket > controller > J CategoriaController.java > CategoriaController
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api/categorias")
14 @Tag(name = "Categorías", description = "Endpoints para administrar categorías de productos")
15 public class CategoriaController {
16
17     @Autowired
18     private CategoriaService categoriaService;
19
20     @GetMapping
21     public List<Categoria> listarCategorias() {
22         return categoriaService.findAll();
23     }
24
25     @GetMapping("/{id}")
26     public ResponseEntity<Categoria> obtenerCategoriaPorId(@PathVariable Long id) {
27         Categoria categoria = categoriaService.findById(id);
28         return (categoria != null) ? ResponseEntity.ok(categoria) : ResponseEntity.notFound().build();
29     }
30
31     @PostMapping
32     public ResponseEntity<Categoria> crearCategoria(@RequestBody Categoria categoria) {
33         return ResponseEntity.ok(categoriaService.create(categoria));
34     }
35
36     @PutMapping("/{id}")
37     public ResponseEntity<Categoria> actualizarCategoria(@PathVariable Long id, @RequestBody Categoria categoria) {
38         return ResponseEntity.ok(categoriaService.update(id, categoria));
39     }
40
41     @DeleteMapping("/{id}")
42     public ResponseEntity<Categoria> eliminarCategoria(@PathVariable Long id) {
43         return ResponseEntity.ok(categoriaService.delete(id));
44     }
45 }
```

Imagen de la representación de las anotaciones en OpenAPI usando Swagger



- Se preparó el escenario para el uso de HATEOAS, con el objetivo de mejorar la navegabilidad entre los recursos.

Se inició la implementación de HATEOAS agregando la dependencia a nuestro archivo POM.XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-hateoas</artifactId>
</dependency>
```

Y se va a usar un enfoque para que éste proyecto envuelva respuestas individuales en con *EntityModel* y colecciones con *CollectionModel* Esto nos va a permitir agregar enlaces como *self*, *collection*, *update* y *delete*.

Ejemplo:

```
private EntityModel<Producto> toModel(Producto producto) {
    EntityModel<Producto> model = EntityModel.of(producto);
    model.add(linkTo(methodOn(ProductoController.class).obtenerProductoPorId(producto.getId())).withSelfRel());
    model.add(linkTo(methodOn(ProductoController.class).listarProductos()).withRel("productos"));
    return model;
}
```

En términos de calidad técnica, este avance transforma el backend desde una API funcional hacia una API mejor documentada, más fácil de consumir y mas clara para integraciones externas.

2. Configuración del entorno de desarrollo

- Detalla los pasos clave para configurar las dependencias necesarias en el proyecto (OpenAPI, HATEOAS).
- Anexa una captura de la estructura de carpetas y del archivo `pom.xml` o equivalente, mostrando las dependencias utilizadas.

Los pasos claves para configurar las dependencias necesarias para el proyecto, fueron la implementación en nuestro archivo POM.XML de las dependencias necesarias para que se pueda ejecutar estos archivos.

Implementación de la dependencia OPENAPI	45 46 47 48 49	<pre><dependency> <groupId>org.springdoc</groupId> <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId> <version>2.6.0</version> </dependency></pre>
Implementación de la dependencia HATEOAS		<pre><dependency> <groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-hateoas</artifactId> </dependency></pre>

Se definió una configuración global de OpenAPI con título, versión, descripción y contacto, Además se utilizaron las anotaciones `@Tag`(Para agrupar funcionalmente), `@Operation`(Para describir la intención de cada método) y `@ApiResponse`(Para declarar respuestas esperadas y errores) en los controladores de Productos, Carritos, Usuarios, Categorías e Inventario.

Durante la implementación de estos, se encontraron algunas dificultades que es necesarias indicarlas.

- A- La primera dificultad que se nos presentó al momento de realizar este paso fue ver que la dependencia de Springdoc no estaba implementada, lo cual impidió que inicialmente pudiéramos visualizar la documentación en Swagger UI y exportar el JSON a OpenAPI.
- B- La segunda dificultad que es importante documentar, se presentó en la seguridad, ya que el backend protege por defecto casi todos los recursos. Para resolverlo, se permitieron explícitamente las rutas de documentación:
`/swagger-ui/**`
`/swagger-ui.html`
`/v3/api-docs/**`
- C- También se observó que las respuestas de algunos controladores no eran homogenas, ya que ciertos métodos devolvían `List<T>`, mientras otros usan

ResponseEntity<T>. Eso rompía la API, pero si nos obligó a documentar con más cuidado los códigos de respuesta.

Respecto a HATEOAS se preparó el proyecto para incorporar HATEOAS en las respuestas JSON, iniciando con la implementación de las dependencias (antes descritas), esto permitió que se pudieran devolver @EntityModel y colecciones con @CollectionModel

Implementación de UsuarioController.java

```
private EntityModel<Usuario> toModel(Usuario usuario) {  
    EntityModel<Usuario> model = EntityModel.of(usuario);  
    model.add(linkTo(methodOn(UsuarioController.class).obtenerUsuarioPorId(usuario.getId())).withSelfRel());  
    model.add(linkTo(methodOn(UsuarioController.class).listarUsuarios()).withRel("usuarios"));  
    return model;  
}
```

Implementación de CarritoController.java

```
private EntityModel<Carrito> toModel(Carrito carrito) {  
    EntityModel<Carrito> model = EntityModel.of(carrito);  
    model.add(linkTo(methodOn(CarritoController.class).obtenerCarritoPorId(carrito.getId())).withSelfRel());  
    model.add(linkTo(methodOn(CarritoController.class).listarCarrito()).withRel("carrito"));  
    return model;  
}
```

Implementación de ProductoController.java

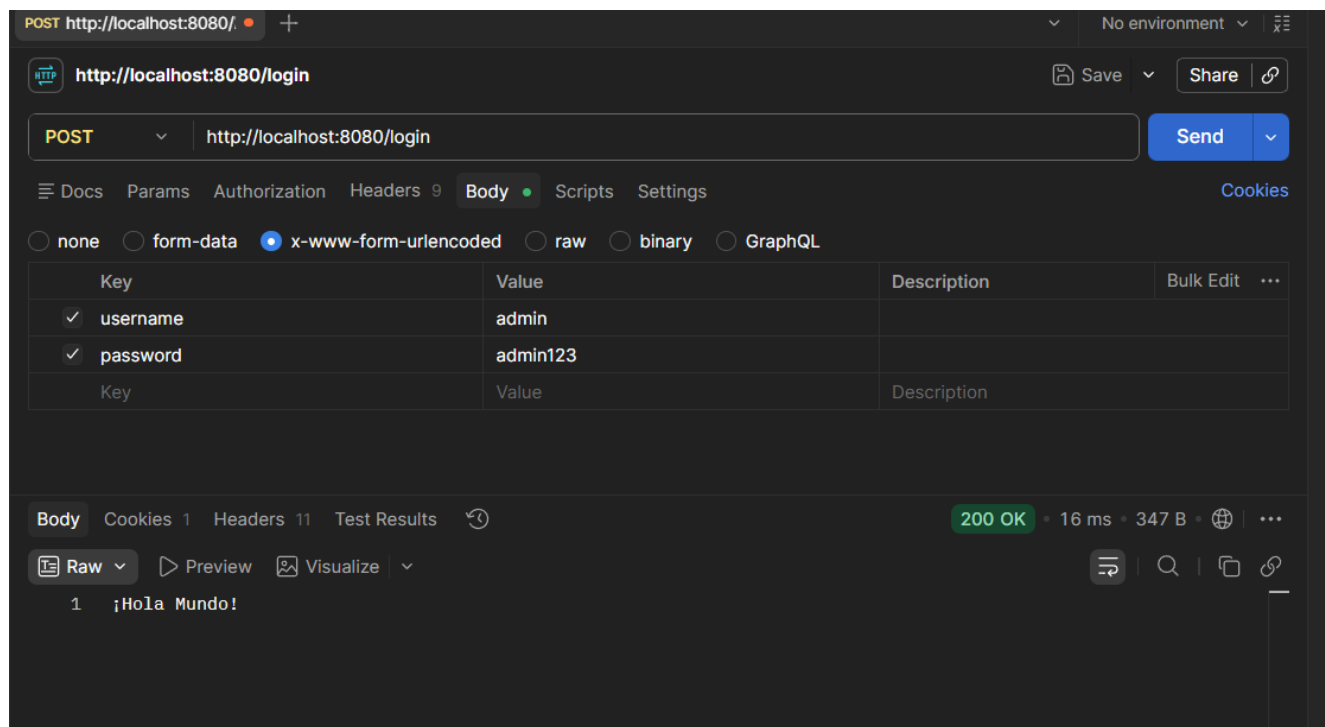
```
}  
  
private EntityModel<Producto> toModel(Producto producto) {  
    EntityModel<Producto> model = EntityModel.of(producto);  
    model.add(linkTo(methodOn(ProductoController.class).obtenerProductoPorId(producto.getId())).withSelfRel());  
    model.add(linkTo(methodOn(ProductoController.class).listarProductos()).withRel("productos"));  
    return model;  
}
```

3. Implementación de HATEOAS

- Muestra ejemplos de respuestas JSON con enlaces dinámicos generados por HATEOAS en los endpoints principales.
- Incluye una captura o fragmento de código que demuestre cómo se añadieron los enlaces utilizando `EntityModel`.

Las pruebas de las respuestas JSON y los enlaces dinámicos de HEATEOAS, fueron realizadas utilizando la herramienta POSTMAN, esto se debió a que como la aplicación requiere de el ingreso de un usuario y contraseña, no supe como agregar esos campos en SWAGGER razón por la cual las capturas serán de esta herramienta:

En POSTMAN con la aplicación ya ejecutándose, lo primero que realizamos fue seleccionar el método POST, posteriormente se agregó la siguiente ruta <http://localhost:8080/login>, después nos fuimos a Body y seleccionamos x-www-form-urlencoded y agregamos en la columna Key username y password y donde dice Value se agregó admin y admin123, finalizamos dándole clic al botón Send y nos devolvió una respuesta 200 Ok y un Hola mundo en consola, lo cual nos permite saber que se ha logeado de forma correcta nuestra aplicación.



PRUEBA DE LOS ENDPOINTS

Productos GET `/api/productos`

GET http://localhost:8080/a

No environment

+

http://localhost:8080/api/productos

Save

Share

New Cl

GET

http://localhost:8080/api/productos

Send

Docs

Params

Authorization

Headers 9

Body •

Scripts

Settings

Cookies

x-www-form-urlencoded

Key	Value	Description	Bulk Edit	...
☒ username	admin			
☒ password	admin123			
Key	Value	Description		

Body

200 OK • 304 ms • 886 B •

{ } JSON

Preview

Visualize

```
1 {
2   "_embedded": {
3     "productoList": [
4       {
5         "id": 1,
6         "nombre": "Leche Entera",
7         "precio": 1290.0,
8         "stock": 25,
9         "categoria": {
10          "id": 1,
11          "nombre": "Lacteos"
12        },
13        "_links": {
14          "self": {
15            "href": "http://localhost:8080/api/productos/1"
16          },
17          "productos": {
18            "href": "http://localhost:8080/api/productos"
19          }
20        }
21      },
22      {
23        "id": 2,
24        "nombre": "Arroz 1kg",
25        "precio": 1390.0,
26        "stock": 40,
27        "categoria": {
28          "id": 2,
29          "nombre": "Abarrotes"
30        },
31        "_links": {
32          "self": {
33            "href": "http://localhost:8080/api/productos/2"
34          },
35          "productos": {
36            "href": "http://localhost:8080/api/productos"
37          }
38        }
39      }
40    ]
41  },
42  "_links": {
43    "self": {
44      "href": "http://localhost:8080/api/productos"
45    }
46  }
47 }
```

GET /api/productos Listar productos

Obtiene todos los productos registrados

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/productos' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/productos

Server response

Code

Details

200

Response body

```
{
  "_embedded": {
    "productolist": [
      {
        "id": 1,
        "nombre": "Leche Entera",
        "precio": 1290,
        "stock": 25,
        "categoria": {
          "id": 1,
          "nombre": "Lacteos"
        }
      },
      {
        "id": 2,
        "nombre": "Arroz 1kg",
        "precio": 1390,
        "stock": 60,
        "categoria": {
          "id": 2,
          "nombre": "Abarrotes"
        }
      }
    ]
  },
  "links": {
    "self": {
      "href": "http://localhost:8080/api/productos/1"
    },
    "productos": {
      "href": "http://localhost:8080/api/productos"
    }
  }
}
```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/hal+json
date: Sun, 12 Jul 2026 15:34:03 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Productos GET /api/productos/{id}

GET http://localhost:8080/a

No environment

HTTP

http://localhost:8080/api/productos/1

Save

Share

GET

http://localhost:8080/api/productos/1

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

x-www-form-urlencoded

Key	Value	Description	Bulk Edit	...
☒ username	admin			
☒ password	admin123			
Key	Value	Description		

Body

200 OK • 587 ms • 563 B •

JSON

Preview

Visualize

```
1  {
2    "id": 1,
3    "nombre": "Leche Entera",
4    "precio": 1290.0,
5    "stock": 25,
6    "categoria": {
7      "id": 1,
8      "nombre": "Lacteos"
9    },
10   "_links": {
11     "self": {
12       "href": "http://localhost:8080/api/productos/1"
13     },
14     "productos": {
15       "href": "http://localhost:8080/api/productos"
16     }
17   }
18 }
```

Productos Endpoints para administrar productos del minimarket

GET /api/productos/{id} Buscar producto por ID

Retorna un producto específico según su identificador

Parameters

Cancel

Name Description

id * required

integer(\$int64)

(path)

1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/productos/1' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/productos/1

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "nombre": "Leche Entera",
  "precio": 1290,
  "stock": 25,
  "categoria": {
    "id": 1,
    "nombre": "Lacteos"
  },
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/productos/1"
    },
    "productos": {
      "href": "http://localhost:8080/api/productos"
    }
  }
}
```



Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/hal+json
date: Sun, 12 Jul 2026 15:30:31 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

Carrito GET /api/carrito

http://localhost:8080/api/carrito

GET http://localhost:8080/api/carrito

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

Key	Value	Description	Bulk Edit	...
✓ username	admin			
✓ password	admin123			
Key	Value	Description		

Body 200 OK · 44 ms · 1.22 KB

JSON Preview Visualize

```
1 {
2   "_embedded": {
3     "carritoList": [
4       {
5         "id": 1,
6         "usuario": {
7           "id": 2,
8           "username": "cliente",
9           "password": "$2a$10$zxCpmvzG0gWe2hVHkjS9Ke4k58Gs7Ph7cvJChNP.r31b.Oo31TKp2",
10          "roles": [
11            {
12              "id": 2,
13              "nombre": "ROLE_CLIENTE"
14            }
15          ]
16        },
17        "producto": {
18          "id": 1,
19          "nombre": "Leche Entera",
20          "precio": 1290.0,
21          "stock": 25,
22          "categoria": {
23            "id": 1,
24            "nombre": "Lacteos"
25          }
26        },
27        "cantidad": 2,
28        "_links": {
29          "self": {
30            "href": "http://localhost:8080/api/carrito/1"
31          },
32          "carrito": {
33            "href": "http://localhost:8080/api/carrito"
34          }
35        }
36      },
37      {
38        "id": 2,
39        "usuario": {
40          "id": 2,
41          "username": "cliente",
42          "password": "$2a$10$zxCpmvzG0gWe2hVHkjS9Ke4k58Gs7Ph7cvJChNP.r31b.Oo31TKp2",
43          "roles": [
44            {
45              "id": 2,
46              "nombre": "ROLE_CLIENTE"
47            }
48          ]
49        },
50        "producto": {
51          "id": 2,
52          "nombre": "Arroz 1kg",
53          "precio": 1390.0,
54          "stock": 40,
55          "categoria": {
56            "id": 2,
57            "nombre": "Abarrotes"
58          }
59        },
60        "cantidad": 1,
61        "_links": {
62          "self": {
63            "href": "http://localhost:8080/api/carrito/2"
64          },
65          "carrito": {
66            "href": "http://localhost:8080/api/carrito"
67          }
68        }
69      }
70    ]
71  },
72  "_links": {
73    "self": {
74      "href": "http://localhost:8080/api/carrito"
```

Carrito

Endpoints para gestionar los productos agregados al carrito

GET /api/carrito/{id} Buscar item del carrito por ID

PUT /api/carrito/{id} Actualizar carrito

DELETE /api/carrito/{id} Eliminar item del carrito

GET /api/carrito Listar carrito

Obtiene todos los items del carrito

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/carrito' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/carrito

Server response

Code Details

200

Response body

```
{
  "producto": {
    "id": 1,
    "nombre": "Arroz 1kg",
    "precio": 1390,
    "stock": 40,
    "categoria": {
      "id": 1,
      "nombre": "Abarrotes"
    }
  },
  "cantidad": 1,
  "links": {
    "self": {
      "href": "http://localhost:8080/api/carrito/2"
    },
    "carrito": {
      "href": "http://localhost:8080/api/carrito"
    }
  }
},
{
  "links": {
    "self": {
      "href": "http://localhost:8080/api/carrito"
    }
  }
}
```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/hal+json
date: Sun, 12 Jul 2026 15:37:52 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Carrito GET /api/carrito/{id}

GET http://localhost:8080/a + No environment

http://localhost:8080/api/carrito/2 Save Share

GET http://localhost:8080/api/carrito/2 Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

☐ none ☐ form-data ☒ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value	Description	Bulk Edit	...
✓ username	admin			
✓ password	admin123			
Key	Value	Description		

Body 200 OK • 56 ms • 746 B

{ } JSON Preview Visualize

```
1 {
2   "id": 2,
3   "usuario": {
4     "id": 2,
5     "username": "cliente",
6     "password": "$2a$10$zxCPmvzG0gWe2hVhkjS9Ke4k58Gs7Ph7cvJChNP.r31b.0o31TKp2",
7     "roles": [
8       {
9         "id": 2,
10        "nombre": "ROLE_CLIENTE"
11      }
12    ]
13  },
14  "producto": {
15    "id": 2,
16    "nombre": "Arroz 1kg",
17    "precio": 1390.0,
18    "stock": 40,
19    "categoria": {
20      "id": 2,
21      "nombre": "Abarrotes"
22    }
23  },
24  "cantidad": 1,
25  "_links": {
26    "self": {
27      "href": "http://localhost:8080/api/carrito/2"
28    },
29    "carrito": {
30      "href": "http://localhost:8080/api/carrito"
31    }
32  }
33 }
```

Carrito Endpoints para gestionar los productos agregados al carrito

GET /api/carrito/{id} Buscar item del carrito por ID

Retorna un item específico del carrito

Parameters

Cancel

Name	Description
------	-------------

id * required

integer(\$int64)

(path)

2

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  http://localhost:8080/api/carrito/2' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/carrito/2
```

Server response

Code	Details
------	---------

200

Response body

```
{
  "id": 2,
  "username": "cliente",
  "password": "$2a$10$zxPmvrG@gle2hVhkj59Ke4k586s7Ph7cvJCHNP.r31b.0o31TKp2",
  "roles": [
    {
      "id": 2,
      "nombre": "ROLE_CLIENTE"
    }
  ],
  "producto": {
    "id": 2,
    "nombre": "Arroz 1kg",
    "precio": 1390,
    "stock": 40,
    "categoria": {
      "id": 2,
      "nombre": "Abarrotes"
    }
  },
  "cantidad": 1,
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/carrito/2"
    },
    "carrito": {
      "href": "http://localhost:8080/api/carrito"
    }
  }
}
```



Download

Usuario GET /api/usuarios

REST client interface showing a GET request to `http://localhost:8080/api/usuarios`. The request is successful (200 OK) and the response is a JSON array of two users.

Request:

- Method: GET
- URL: `http://localhost:8080/api/usuarios`
- Body: none

Response:

```
1 {
2   "_embedded": {
3     "usuarioList": [
4       {
5         "id": 1,
6         "username": "admin",
7         "password": "$2a$10$mytqAb0rIQs3KdGgg8PZsuXtsS/K0KBaEU5m09dwTVYPuyB2kYaqS",
8         "roles": [
9           {
10            "id": 1,
11            "nombre": "ROLE_ADMIN"
12          }
13        ],
14        "_links": {
15          "self": {
16            "href": "http://localhost:8080/api/usuarios/1"
17          },
18          "usuarios": {
19            "href": "http://localhost:8080/api/usuarios"
20          }
21        }
22      },
23      {
24        "id": 2,
25        "username": "cliente",
26        "password": "$2a$10$zxCpmvzG0gWe2hVHkjS9Ke4k58Gs7Ph7cvJChNP.r3lb.0o31TKp2",
27        "roles": [
28          {
29            "id": 2,
30            "nombre": "ROLE_CLIENTE"
31          }
32        ],
33        "_links": {
34          "self": {
35            "href": "http://localhost:8080/api/usuarios/2"
36          },
37          "usuarios": {
38            "href": "http://localhost:8080/api/usuarios"
39          }
40        }
41      }
42    ]
43  },
44  "_links": {
45    "self": {
46      "href": "http://localhost:8080/api/usuarios"
47    }
48  }
49 }
```

Usuarios Endpoints para administrar usuarios del sistema

GET /api/usuarios/{id} Buscar usuario por ID

PUT /api/usuarios/{id} Actualizar usuario

DELETE /api/usuarios/{id} Eliminar usuario

GET /api/usuarios Listar usuarios

Obtiene todos los usuarios registrados en el sistema

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/usuarios' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/usuarios

Server response

Code Details

200

Response body

```
{
  "_embedded": {
    "usuarioList": [
      {
        "id": 1,
        "username": "admin",
        "password": "$2a$10$mytgAbOrIQs3KdGgg8PZsuXts5/K0KBaEU5wO9dwTVYPuy82KYaq5",
        "roles": [
          {
            "id": 1,
            "nombre": "ROLE_ADMIN"
          }
        ],
        "links": {
          "self": {
            "href": "http://localhost:8080/api/usuarios/1"
          },
          "usuarios": {
            "href": "http://localhost:8080/api/usuarios"
          }
        }
      },
      {
        "id": 2,
        "username": "cliente",
        "password": "$2a$10$zxCPmvzG8gMe2HWHKjS9Ke4K58Gs7Ph7cv3CHMP.r31b.0o31TKp2",
        "roles": [
          {
            "id": 2,
            "nombre": "ROLE_CLIENTE"
          }
        ]
      }
    ]
  }
}
```



Download

Usuario GET /api/usuarios/{id}

HTTP

http://localhost:8080/api/usuarios/1

Save

Share

GET

http://localhost:8080/api/usuarios/1

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

Key	Value	Description	Bulk Edit	...
☒ username	admin			
☒ password	admin123			
Key	Value	Description		

Body

200 OK • 48 ms • 603 B •

{}

JSON

Preview

Visualize

1

{

2

"id": 1,

3

"username": "admin",

4

"password": "\$2a\$10\$mytqAb0rIQs3KdGgg8PZsuXtsS/K0KBaEU5m09dwTVYPuyB2kYaqS",

5

"roles": [

6

{

7

"id": 1,

8

"nombre": "ROLE_ADMIN"

9

}

10

],

11

"_links": {

12

"self": {

13

"href": "http://localhost:8080/api/usuarios/1"

14

},

15

"usuarios": {

16

"href": "http://localhost:8080/api/usuarios"

17

}

18

}

19

}

Usuarios

Endpoints para administrar usuarios del sistema

GET /api/usuarios/{id} Buscar usuario por ID

Retorna un usuario específico según su identificador

Parameters

Cancel

Name Description

id * required
integer(\$int64)
(path)

1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/usuarios/1' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/usuarios/1

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "username": "admin",
  "password": "$2a$10$mytqAb0rIQs3Kdgg8PZsuKtsS/K0K8aEUSw09dwTVYPuy62kYaqS",
  "roles": [
    {
      "id": 1,
      "nombre": "ROLE_ADMIN"
    }
  ],
  "links": {
    "self": {
      "href": "http://localhost:8080/api/usuarios/1"
    }
  },
  "usuarios": {
    "href": "http://localhost:8080/api/usuarios"
  }
}
```



Download

Reflexión técnica sobre calidad, navegabilidad y mantenibilidad del backend

Como reflexión podemos indicar que la combinación de OpenAPI y HATEOAS mejora la calidad del backend porque documenta y orienta al mismo tiempo. OpenAPI establece el contrato técnico, mientras que HATEOAS guía la navegación entre recursos en tiempos de ejecución.

Desde el punto de vista de mantenibilidad, esto reduce el costo de onboarding de nuevos desarrolladores, hace más predecibles los cambios y facilita la validación de versiones futuras. Desde el punto de vista de navegación, los enlaces evitan que el consumidor tenga que memorizar rutas o dependencias de documentación externa para continuar una operación.

En definitiva, esta implementación en el backend queda mejor preparada para crecer sin perder la claridad ni consistencia.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.